Report

PG4200-H: Data Structures and Algorithms

Group number: 11
Candidate: 264 and 140

Contents

Introduction 2
- Unique alcohol content values 2
- JVM JIT 3
- Algorithms 4
  - Computational complexity 4
  - Algorithmic Operations 5
- Data Structures 5
  - Non-primitive static Data Structure - Array 7
  - Array Operations 7

Problem 1 – BubbleSort 10
- Usage 10
- Implementation 10
- Visualization 12
  - Unoptimized 12
  - Optimized 12
- Mean average time usage 13
- Computational complexity 14
  - Time Complexity 14
  - Space complexity 14
- When to use 14

Problem 2 – InsertionSort 16

Implementation 16
Visualization 17
Mean average time usage 17
Computational complexity 19
Time complexity 19
Space Complexity 20
When to use 21
Problem 3 – MergeSort 22
Usage 22
Implementation 22
Visualization 25
Mean average time usage 27
Computational complexity 27
Time Complexity 27
Space complexity 28
When to use 28
Problem 4 – QuickSort 29
Implementation 29
Sorting 29
Partition 30
Swap 32
Pivot 32
Visualization 34
Mean average time usage, Swaps, and comparisons 35
Computational complexity 37
Time Complexity 37
Space Complexity 37
When to use 37
References 39

Introduction

Unique alcohol content values

In this exam we have been specifically instructed to use unique alcohol content values in each of the exercises and then use all alcohol content values in the algorithms (Gupta, 2026f). We understand this as only using unique values and run all the unique alcohol values on the algorithms.

To achieve this, we read the red and white wine csv files, combine them into a list, passed through a LinkedHashSet to remove duplicates and preserve insertion order, convert to a Double array with toArray(). In the algorithms we return a copy of the array and not the original array.

Visualization:

1. Combine red and white csv lists

```java
    public static List<Double> loadAlcoholValues() throws IOException {
        List<Double> wineList = new ArrayList<>();

        //redList
        for (double value : readFile(redFile)) {
            wineList.add(value);
        }

        //whiteList
        for (double value : readFile(whiteFile)) {
            wineList.add(value);
        }

        return wineList;
    }
}
```

2. Pass through LinkedHashSet to remove duplicates and preserve insertion order

```java
Set<Double> alcoholValue = new LinkedHashSet<>(WineReader.loadAlcoholValues());
```

3. Convert to Double array with toArray()

```java
Double[] array = alcoholValue.toArray(new Double[0]);
```

4. Copy of array

```java
Double[] a = Arrays.copyOf(x, x.length);
```

The easiest way to remove duplicates is using HashSet, TreeSet or LinkedHashSet. Since this is a data and algorithm exam which requires different algorithms to be coded and run with a list containing unique values, we made the decision to require a data structure that does not pre-sort and returns unique values.

HashSet does not maintain insertion or sorting order and returns unique values.

TreeSet pre-sorts values in a natural order or with a custom comparator and returns unique values.

LinkedHashSet preserves insertion-order from the combined csv file and returns unique values (rajput-ji, n.d.).

We chose to use LinkedHashSet because of its capabilities to preserve insertion order and return unique values. This enables the algorithms to be run with arrays that are not pre-sorted which further enables us to calculate the correct time on non-sorted arrays and randomized arrays (rajput-ji, n.d.).

JVM JIT

In this course, we are required to use JetBrains IntelliJ IDEA and a Java Development Kit (JDK) which is implemented in IntelliJ (*Subject Description PG4200 Algorithms and Data Structures*, n.d.). By using the software required, we are bound to restrictions and set rules provided by the software. The JDK Java Virtual Machine's Just In Time compiler must be warmed up and run once or twice to return proper time used. Results with and without warmup can differ by 1 to 2 seconds.
This means that we need to take precautions when calculating time used for each algorithm by ignoring the first one or two algorithmic runs, if not we will get "false" time used (priyavermaa1198, n.d.).

Visualization by using BubbleSort Unoptimized:
The time used by the algorithm will vary and is not set in stone.

- Not warmed up

```
BubbleSort Unoptimized:
[8.0, 8.4, 8.5, 8.6, 8.7,
Seconds used: 1.5981
```

- Warmed up, 1st run

```
BubbleSort Unoptimized:
[8.0, 8.4, 8.5, 8.6, 8.
Seconds used: 0.8022
```

- Warmed up, 2nd run

```
BubbleSort Unoptimized:
[8.0, 8.4, 8.5, 8.6, 8.7
Seconds used: 0.8536
```

- Warmed up, 3rd run

```
BubbleSort Unoptimized:
[8.0, 8.4, 8.5, 8.6, 8.7
Seconds used: 0.6501
```

Algorithms

An algorithm is used to process data meaningfully and efficiently (Gupta, 2026a, p. 4). In this exam we have been tasked to implement four algorithms: Bubble Sort, Insertion Sort, Merge Sort and Quick Sort. In our solution to this exam, we have implemented these solutions as Java code, but it is important to remember that algorithms are not just code. According to Richard Neopolitan it is an "approach or methodology used to solve a problem" (Neapolitan, 2025, p. 1). This can be explained further by using the phonebook example. Open a phonebook and check for a name, how do you solve this problem step by step? That is an algorithm, a specific solution to a problem.
All algorithms are an approach to a solution for a problem, finding specific values or sorting values in a specific way (Neapolitan, 2025, p. 1).

Computational complexity

Algorithms are evaluated by time complexity and space complexity. This is called Computational complexity and was developed by Juris Hartmanis and Richard E. Steams in 1965 (Gupta, 2026d, p. 3).

By using time complexity, we can calculate how much time that it is required to execute an algorithm. This calculation has three "cases" with a notation type. It is important to note that time complexity does not measure how much time it takes to run an algorithm, since this is based on operating system, code language and processing power (Gupta, 2026d, p. 4).

1. Best case – Omega notation (Ω) – Least time used
2. Average case – Theta notation (Θ) – Average time used
3. Worst case – Big-O notation (O) – Most time used

(Neapolitan, 2025, p. 19)

According to Richard Neopolitian, time complexity "is the determination of how many times the basic operation is done for each value of the input size" (Neapolitan, 2025, p. 19).
This means that it is possible to count how long an algorithm will take based on each pass the code takes.
There are 7 types of time complexity:

1. Constant time – $O(1)$
2. Logarithmic Time – $O(log\ n)$
3. Linear Time – $O(n)$
4. Linear Logarithmic Time – $O(n\ log\ n)$
5. Quadriatic/Polynomial Time – $O(n^2)$
6. Exponential Time – $O(2^n)$
7. Factorial Time – $O(n!)$

(Gupta, 2026d, p. 8)

Space complexity is a way to calculate how memory efficient an algorithm is and how much memory it would need in addition to the data value being manipulated with the algorithm. It is

important to note that space complexity is not the memory needed to store the input but only the memory needed to run the algorithm and execute its needed functions (Gupta, 2026d, p. 6; Neapolitan, 2025, p. 24).

Algorithmic Operations

Major operations in algorithms allow it to run key functions that are required to fulfill the specific algorithm's purpose. The major operations are:

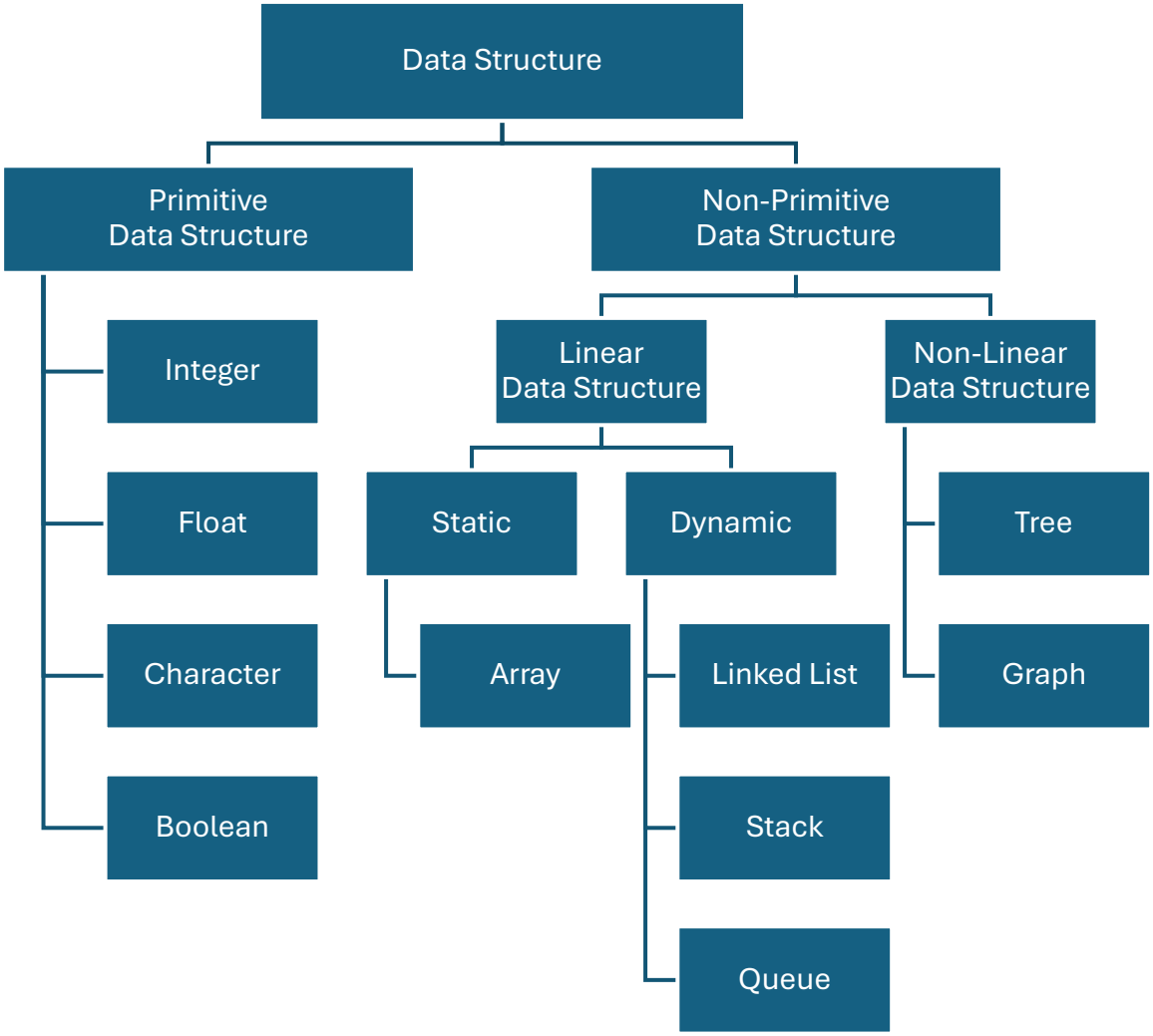
- Search
  Allows the algorithm to find the data value requested.
- Sort
  Arranges data values in a certain order decided by the pre-determined rules in an algorithm.
- Insert
  Allows the algorithm to add new data values to the data structure.
- Update
  Allows the algorithm to change outdated data values with up-to-date data values.
- Delete
  Allows the algorithm to remove data values that are not required anymore.

(Gupta, 2026a, p. 9, 2026b)

Data Structures

If we or a user needs to access data and/or easily find it, we need to organize and store it properly. This is where data structures come in, it allows us to organize and store data meaningfully. In short, data needs to be structured to be processed efficiently.
There are two main categories in data structures: Primitive and non-primitive. Data that can be found in the primitive part of data structures are int, double, char and Boolean. They are, as in the word, "primitive". They hold single values and are stored directly in memory (Gupta, 2026b, 2026a, p. 4).
The Non-primitive category consists of one or more primitive data structures in specific orders and with certain rules attached to them. There are two categories: Linear and Non-Linear data structures. Linear consists of two "under"-categories called Static and Dynamic. Static consists of an Array while dynamic consists of Linked List, Stack, and Queue. Non-Linear consists of Tree and Graph structures (Gupta, 2026b, 2026a, p. 8).

Visualization of Data Structure:



(Gupta, 2026a, p. 8)

Non-primitive static Data Structure - Array

In our solution to this exam's problems, we have decided to use Array, specifically Double Array. We have used double to make sure we properly read all decimals needed from the white and red csv files. Array is used since its index based and is suited for sorting algorithms since it requires swapping and comparisons of elements by index. The exams problems are focused on sorting algorithms, which is our main reason for the use of array.

Array allows for collecting one data type, which must be the same and is stored at contiguous memory locations in the hardware. If there was a gap in the memory location of the data values, it would not be an array but could be a different data structure like a linked list. The memory location of each data value can be found by calculating the base's address and size of the data element. The index location starts at 0 and is +1 for each data value (Gupta, 2026c, p. 3).

Example of array consisting of int's:
Base address – 250
Int size – 4 bytes

Data value	1	2	3	4
Index location	0	1	2	3
Memory location	250	254	258	262

Based on the array above we can see that it is stored in continuous memory location

Example of an array with multiple different data values, which is not a valid array:

Data value	1	"Algorithm"	2.1	false
Index	0	1	2	3
Memory location	250	254	258	262

Array Operations

Major operations for array data structures are search, sort, insert, update and delete. These are key functions that allows algorithms to carry out different operations when interacting with an array and the data values in said array (Gupta, 2026c, p. 5).
Code examples below use examples from our own code.

- Search
  Allows for finding specific data values requested by the user.
  Code example:
  QuickSort.java and partition method for loop. This code iterates through a set array to find elements connected to a pivot point.

```
for (int i = first; i < last; i++) {
    Counter.numberOfComparisons++;
    if (a[i] < pivot) {
        n++;
        if (i != n) {
            swap(a, i, n);
            Counter.numberOfSwaps++;
        }
    }
}
```

- Sort
  Allows for changing the position of data values to match algorithmic requirements requested by the user.
  This is the main requirement in our exam's problems 1, 2, 3 and 4, specifically Bubble Sort, Insertion Sort, Merge Sort and Quick Sort.

  Code example:
  BubbleSort.java and optimizedBubbleSort method. This code compares adjacent elements and swaps them into the correct order for BubbleSort.

```
if (a[j] > a[j + 1]) {
    double temp = a[j];
    a[j] = a[j + 1];
    a[j + 1] = temp;
    swapped = true;
    Counter.numberOfSwaps++;
}
```

- Insert
  Allows for adding new data values to the data structure.

  Code example:
  MergeSort.java and merge method. This code sets a[index k] to be temporary array named arrayRight[index j] and increments j which is used for arrayRight index.

```
else {
    a[k] = arrayRight[j];
    j++;
}
```

- Update
  Allows for changing outdated data values with up-to-date data values.

  Code example:
  InsertionSort.java and sort method. This code sets a[index+1] to be the value of a[index]

```
if (a[index] > temp){
    a[index+1] = a[index];
    index--;
    Counter.numberOfSwaps++;
} else {break;}
```

- Delete
  Allows for removing data values in the data structure.

  Code example:
  We do not have deletion code in our algorithms which could be used as a clear example. However, this is a general example that uses the example array in " Non-primitive static Data Structure - Array".

```
int n = array.length;

for (int i = 1; i < n - 1; i++) {
    array[i] = array[i +1];
}
array[n – 1] = 0;
```

(Gupta, 2026c, p. 5).

Problem 1 – BubbleSort

Bubble Sort is a simple algorithm that repeatedly swaps adjacent elements until every element is in a specific order, this order could be ascending or descending. This algorithm swaps and compares elements. Swapping means that it switches the position of two elements with each other. Comparison means that two elements' values are evaluated against each other's values and checked if they are the same, above or below the other value.
If the compared elements are in the wrong order, they are swapped and then the algorithm goes on to the next pair of adjacent and comparable elements. This is one of the "easiest" algorithms to understand and implement but it is slow for larger datasets (Gupta, 2026e, p. 3; kartik, n.d.-a).

Usage

Bubble Sort works by comparing each adjacent element of the index location and if the inspected element is in the wrong order, it will swap the elements (kartik, n.d.-a). Let's use 1 4 3 2 as an example, it will compare 1 against 4 and determine if it is in the wrong order. If it is in the wrong order, it will swap them. Since they are not in the wrong order when sorting increasingly, they will compare the 2nd element against the 3rd element which is 4 and 3. This continues until all values are in increasing or decreasing order.

Implementation

In the start of our code, the array being sent to the algorithm is copied and returned in the end with "return a". It is also worth noting that n is set to the number length of the array.

The unoptimized version will compare every value in the example array, even though it has already been compared. This means that the whole array will be compared even if the whole array is already sorted where it will perform unnecessary comparisons with no swaps.
Example from the attached code:

1. For loop that controls the number of passes through the whole array.
   ```
   for (int i = 0; i < n - 1; ++i) {
   ```
   It will run for n – 1. (length of array – 1)
2. For loop that compares each adjacent value pair.
   ```
   for (int j = 0; j < n - i - 1; ++j) {
   ```
   It will iterate through each unsorted element and then compare adjacent value pairs. After each comparison, the comparison counter variable is increased.
   ```
   Counter.numberOfComparisons++;
   ```

3. Swaps adjacent value pairs if the left element is greater than the right.

```
if (a[j] > a[j + 1]) {
    double temp = a[j];
    a[j] = a[j + 1];
    a[j + 1] = temp;
```

Temporary variable "temp" is used for this swap.

```
a[j + 1] = temp;
```

After each swap, the swap counter variable is increased.

```
Counter.numberOfSwaps++;
```

The optimized version uses a "swapped" Boolean, but this can be any sort of code snippet that works as a "flag" the code can detect changes too. This version compares every value like bubble sort usually does but will change "swapped" to "true" if it executes a swap. If it does not execute a swap, "swapped" is not changed and will trigger a break out of the for loop.

The attached code works in the same way as unoptimized. The changes are that "swapped" Boolean is set to false at the start when each pass is done, "swapped" Boolean is set to "true" if a swap has occurred and the pass will stop if "swapped" Boolean is still false after a pass.

- "Swapped" Boolean set to false at the start of each pass.
  This is done after Nr 1.

```
boolean swapped = false;
```

- "Swapped" Boolean is set to true if a swap has occurred.
  This is done after the "temp" variable is set in Nr 3.

```
swapped = true;
```

- If statement that checks if "swapped" Boolean is still set to "false" after a pass and breaks the loop. This is done after Nr 3.

```
if (!swapped) {
    break;
}
```

(Gupta, 2026e, p. 3; kartik, n.d.-a)

Visualization

Array used for visualization:

Data values	1	4	3	2
Index	0	1	2	3

Unoptimized

Number example:

Pass	Compared index	Array before	Array after	Swap?
1	0 and 1	1, 4, 3, 2	1, 4, 3, 2	No
1	1 and 2	1, 4, 3, 2	1, 3, 4, 2	4 and 3
1	2 and 3	1, 3, 4, 2	1, 3, 2, 4	4 and 2
2	0 and 1	1, 3, 2, 4	1, 3, 2, 4	No
2	1 and 2	1, 3, 2, 4	1, 2, 3, 4	3 and 2
2	2 and 3	1, 2, 3, 4	No change	No
3	0 and 1	1, 2, 3, 4	No change	No
3	1 and 2	1, 2, 3, 4	No change	No
3	2 and 3	1, 2, 3, 4	No change	No
4	0 and 1	1, 2, 3, 4	No change	No
4	1 and 2	1, 2, 3, 4	No change	No
4	2 and 3	1, 2, 3, 4	No change	No

The end array after sorting is 1, 2, 3, 4. It is finished sorting at pass 2 but since it's unoptimized it will check the whole array.

Optimized

Pass	Compared index	Array before	Array after	Swap?	Sorted position
1	0 and 1	1, 4, 3, 2	1, 4, 3, 2	No	
1	1 and 2	1, 4, 3, 2	1, 3, 4, 2	4 and 3	
1	2 and 3	1, 3, 4, 2	1, 3, 2, 4	4 and 2	Index 3
2	0 and 1	1, 3, 2, 4	No change	No	
2	1 and 2	1, 3, 2, 4	1, 2, 3, 4	3 and 2	Index 2
3	0 and 1	1, 2, 3, 4	No change	No	Index 1
Pass 4	Skipped	-	-	-	-

The end array after sorting is 1, 2, 3, 4. It is finished sorting at pass 3 and is skipping the rest of the passes since the optimized algorithm checks if swapped is set to false and then breaks out of the for loop.

Mean average time usage

When starting the program we have created, 2 random algorithms are run and ignored before using timing to calculate an mean average for each algorithm.

Run	Version	Time (seconds)	Swaps	Comparisons	Mergers
1	Unoptimized	0.9489	2607	6105	0
2	Unoptimized	0.9225	2607	6105	0
3	Unoptimized	1.1093	2607	6105	0
4	Unoptimized	0.7667	2607	6105	0
5	Unoptimized	1.6208	2607	6105	0
	Mean average	1,0736	2607	6105	
1	Optimized	1.4683	2607	5895	0
2	Optimized	0.6773	2607	5895	0
3	Optimized	1.794	2607	5895	0
4	Optimized	1.2649	2607	5895	0
5	Optimized	0.7958	2607	5895	0
	Mean average	1.2001	2607	5895	
1	Unoptimized Randomized	1.0822	3046	6105	0
2	Unoptimized Randomized	0.849201	2632	6105	0
3	Unoptimized Randomized	0.7956	3186	6105	0
4	Unoptimized Randomized	0.7678	3202	6105	0
5	Unoptimized Randomized	0.2916	3120	6105	0
	Mean average	0.7572802	3037,2	6105	0
1	Optimized Randomized	0.6236	3198	5969	0
2	Optimized Randomized	0.6425	3077	6027	0
3	Optimized Randomized	0.1994	3337	6105	0
4	Optimized Randomized	0.604	2996	6000	0
5	Optimized Randomized	0.5961	2743	5934	0
	Mean average	0.53312	3070,2	6007	0

Computational complexity

Time Complexity

Bubble Sort's time complexity is based on the loops in the algorithm. This is called a nested loop structure and consists of an inner and outer loop. The method used to calculate this is n – x where n is the number of elements needed to sort and x is number of passes needed to sort the elements. This is often described as n-(passes) (Animesh, n.d.-a).

Best case is $O(n)$ which is N-1 and requires the array to be already sorted.
Average and worst case is $O(N^2)$ which N-(passes) or N * (N-1)/2 and will run until the array is sorted.
Worst case is also $O(N^2)$ and will also run until the array is sorted with multiple passes.
The calculation of these cases are the same even if it is randomized (Animesh, n.d.-a).

Time complexity for the exam's dataset with only unique values:

Best case	Average case	Worst case
0	111 * 110 /2 = 6105	111 * 110 /2 = 6105

Running optimized or unoptimized on a randomized array will not change the time complexity. The algorithm will still have to sort 111 elements. Average and worst case is still $O(n^2)$ since it is not affected by input order. The unoptimized version will run all N – 1 passes. The optimized version will still possibly exit early if it does not detect a swap in one pass the swap, which is seen in "Mean average time used" above (Animesh, n.d.-a).
So no, the time complexity does not change if you randomly shuffle the list before sorting.

Space complexity

Bubble sort's space complexity is $O(1)$. This algorithm requires little extra memory since it is constant in the sense that it doesn't change with different size arrays. The extra memory it needs is based on the additional temporary variables needed during the sorting process. The original array input is not calculated or used in space complexity it only cares about the extra temporary variables (Animesh, n.d.-a).

When to use

Bubble sort is not recommended to use for larger datasets since it is slow. This can be seen with bubble sort's time complexity. The unoptimized version always executes with $O(N^2)$ while the optimized version can achieve best case $O(n)$ if the array is already sorted. However, if the array is not pre-sorted it will be worst or average case $O(n^2)$.
Since the exams required dataset of unique values is relatively small, it will produce quick results. If the dataset was on bigger datasets, it would be inefficient and we would recommend using quick sort or merge sort. (Animesh, n.d.-a; Gupta, 2026f).

According to GeeksforGeeks, Bubble sort is almost only used to teach students how sorting and algorithms work (kartik, n.d.-a).

Problem 2 – InsertionSort

Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list. It is like sorting playing cards in your hands. You split the cards into two groups: the sorted cards and the unsorted cards. Then, you pick a card from the unsorted group and put it in the right place in the sorted group (kartik, n.d.-b)

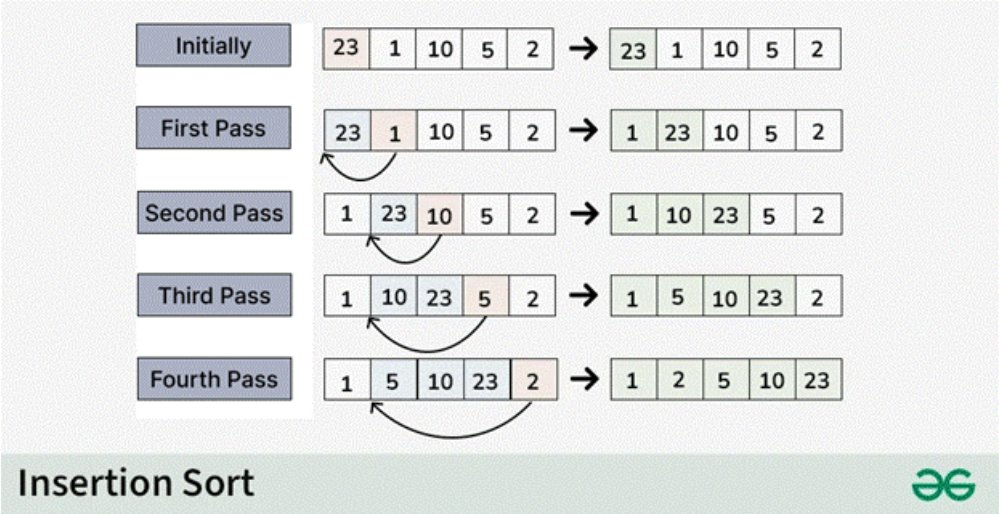


Figure 1 https://www.geeksforgeeks.org/dsa/insertion-sort-algorithm/

Implementation

```
for (int i = 1; i < n; i++) {

    double temp = a[i]; // The value we want to insert in the correct position
    int index = i - 1;  // Start comparing with the element to the left
```

In the start of our code, we create a for loop that runs through the array with data. We then create the variable "temp" which is the element we want to sort. The temporary variable "index" to compare our chosen element to the elements to the left.

```
// the loop wil run true alle the elements left of the index
while (index >= 0) {
```

The while loop checks all elements to the left of the current index. It makes sure that the current element (temp) is compared with every previous element, not just the one next to it. For example, in the array [9, 5, 1], after 5 and 9 are sorted [5,9,1], we insert 1. First, 1 is compared with 9 and is moved [5, 1, 9]. Then 1 will be compared to 5, and placed in the correct position: [1, 5, 9].

```
if (a[index] > temp) {
    a[index + 1] = a[index]; // Move element one step to the right
    index--; // Move left
    Counter.numberOfSwaps++; // Count shifts (treated as swaps)
} else {
    break; // Stop when the correct position is found
}
```

Here is the main part of the code that sorts the elements. If the current element (temp) we will want to sort is smaller than the element to the left (index), The element is then moved to the right. Then we move left in the array to check more elements. The if loop stops when the correct position is found.

Visualization

Array used for visualization:

Data values	1	4	3	2
Index	0	1	2	3

Our insertion sort algorithm starts at the second element at index 1 and compares to the element to the left.

Pass	Current element	Compared element	Array before	Array after	Swap
1	4	1	1, 4, 3, 2	1, 4, 3, 2	No
2	3	4	1, 4, 3, 2	1, 3, 4, 2	4 moved to the right
2	3	1	1, 3, 4, 2	1, 3, 4, 2	No
3	2	4	1, 3, 4, 2	1, 3, 2, 4	4 moved to the right
3	2	3	1, 3, 2, 4	1, 2, 3, 4	3 moved to the right
3	2	1	1, 2, 3, 4	1, 2, 3, 4	No

After sorting the array is 1, 2, 3, 4. This systematically compares elements and moves smaller values to the left by moving larger values to the right.

Mean average time usage

When starting the program we have created, 2 random algorithms are run and ignored before using timing to calculate a mean average for each algorithm.

Run	Version	Time (seconds)	Swaps	Comparisons	Mergers
1	Insertion sort	0.2163	2607	2711	0
2	Insertion sort	0.4061	2607	2711	0
3	Insertion sort	0.4109	2607	2711	0
4	Insertion sort	0.1347	2607	2711	0
5	Insertion sort	0.2105	2607	2711	0
	Mean average	0.2757	2607	2711	0

Run	Version	Time (seconds)	Swaps	Comparisons	Mergers
1	Insertion sort Randomized	0.452	3328	3434	0
2	Insertion sort Randomized	0.492	3240	3348	0
3	Insertion sort Randomized	0.8786	3068	3173	0
4	Insertion sort Randomized	0.3682	2854	2957	0
5	Insertion sort Randomized	0.3360	3465	3569	0
	Mean average	0.41496	3191	3296.2	0

Computational complexity

Time complexity

Best case: O(n), If the list is already sorted, where n is the number of elements in the list (Marculescu, n.d., p. 51).

Example:

Data values	1	2	3	4
Index	0	1	2	3

Pass	Current element	Compared element	Array before	Array after	Swap
1	2	1	1, 2, 3, 4	1, 2, 3, 4	No
2	3	2	1, 2, 3, 4	1, 2, 3, 4	No
3	4	3	1, 2, 3, 4	1, 2, 3, 4	No

The algorithm makes a single comparison per element. Or in other words N-1 comparisons, so O(n).

Average case: $O(n^2)$, if the list is randomly ordered (Gupta, 2026).

Data values	1	4	3	2
Index	0	1	2	3

Pass	Current element	Compared element	Array before	Array after	Swap
1	4	1	1, 4, 3, 2	1, 4, 3, 2	No
2	3	4	1, 4, 3, 2	1, 3, 4, 2	4 moved to the right
2	3	1	1, 3, 4, 2	1, 3, 4, 2	No
3	2	4	1, 3, 4, 2	1, 3, 2, 4	4 moved to the right
3	2	3	1, 3, 2, 4	1, 2, 3, 4	3 moved to the right
3	2	1	1, 2, 3, 4	1, 2, 3, 4	No

Example: like in the array above, the algorithm likely makes multiple comparisons and shifts.

Worst case: $O(n^2)$, if the list is in reverse order (Gupta, 2026).
Example:

Data values	4	3	2	1
Index	0	1	2	3

Pass	Current element	Compared element	Array before	Array after	Swap
1	3	4	4,3,2,1	3,4,2,1	4 moved to the right
1	3	-	3,4,2,1	3,4,2,1	No
2	2	4	3,4,2,1	3,2,4,1	4 moved to the right
3	2	3	3,2,4,1	2,3,4,1	3 moved to the right
3	2	-	2,3,4,1	2,3,4,1	No
4	1	4	2,3,4,1	2,3,1,4	4 moved to the right
4	1	3	2,3,1,4	2,1,3,4	3 moved to the right
4	1	2	2,1,3,4	1,2,3,4	2 moved to the right
4	1	-	1,2,3,4	1,2,3,4	No

It's the worst case for insertion sort since it needs to compare and insert after every element. Each time we insert an element into the sorted portion, we'll need to swap it with each of the elements already in the sorted array to get it all the way to the start. That's 1 swap the first time, 2 swaps the second time, 3 swaps the third time, and so on, up to n(n-1) swaps for the last item. This gives a time complexity of $O(n^2)$ (Brilliant, n.d.; Exponent, n.d.)

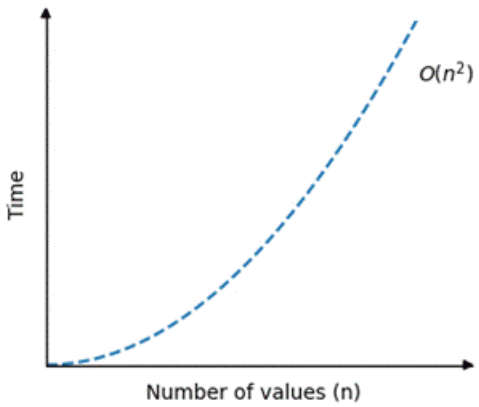


Figure 2 https://www.w3schools.com/dsa/dsa_timecomplexity_insertionsort.php

Space Complexity

The auxiliary space complexity of Insertion Sort is O(1), indicating it uses constant extra space regardless of the input size. This is because the algorithm typically performs in-place sorting, meaning it rearranges the elements within the input array itself without requiring

additional data structures or memory allocation proportional to the input size. Therefore, regardless of the size of the input array, the amount of extra space used by the Insertion Sort algorithm remains constant (kartik, n.d.-b).

When to use

Insertion sort is commonly used in situations where:

The list is small or nearly sorted. Simplicity and stability are important. Since Insertion sort is suitable for small sized arrays, it is used in Hybrid Sorting algorithms along with other efficient algorithms like Quick Sort and Merge Sort. It is hover not recommended for large, unsorted datasets due to its $O(n^2)$ time complexity (kartik, n.d.-b; Noroff, n.d.)

Problem 3 – MergeSort

Merge sort is a divide and conquer algorithm that is known to be efficient and stable. It is called a divide and conquer algorithm because it recursively divides the array into two parts, sorting and then merging them together into one array.

Merge sort is complicated compared to bubble sort but is also very efficient on large datasets and external datasets which gives it more use areas (kartik, n.d.-c).

Usage

Merge Sort works by dividing the input array in half until every element is in its own subarray and then merging them together while sorting each subarray. During the merging, the algorithm compares each merged value and places the smaller element in the next available location. This repeats until all the subarrays are sorted and merged into a fully sorted array. The algorithm will divide the left subarrays first, then move over to the right subarrays, merge the left subarrays, merge the right subarrays and lastly merge both right and left subarray. The divide section is referred to as "Divide" since it splits each element into subarrays while the merging and sorting section is referred to as "Conquer" since it compares (sorts) and places the elements in a certain order (correct order) (kartik, n.d.-c).

Let's use 1 4 3 2 as an example; it will split 1, 4 and 3, 2 into subarrays, then split 1 and 4 into separate subarrays and 3 and 2 into separate subarrays. The next step is comparing and merging 1 and 4 into 1, 4, and compare and merge 3 and 2 into 2, 3. Lastly, compare and merge 1, 4 and 2, 3 into 1, 2, 3, 4.

Implementation

In the start of our code, the array being sent to the algorithm is copied and returned in the end with "return a". It is also worth noting that n is set to the number length of the array. It is also worth noting that merge sort does not swap elements but places them in the available position, which is why "swaps" are not counted.

In this algorithm, there is no optimized or unoptimized like bubble sort's different versions. Merge sort has one version, and one execution except non-randomized and randomized arrays but this has no effect on how the algorithm executes. Please note that capital I and lowercase i is the same variable, this is an issue found while writing in word and is directly a result of word's grammar correction which is out of the users control.
Examples from the attached code:

1. mergeRun is called from the menu.
   The original array is copied, length of the copied array is calculated and mergeSort method is executed with copy array "a", left index started at 0 and right index started at the last element of "a" array.

```
public static Double[] mergeRun(Double[] x) {
    Double[] a = Arrays.copyOf(x, x.length);
    int n = a.length;
    mergeSort(a, indexLeft: 0, indexRight: n-1);
```

The sorted array "a" is returned to the menu and printed.

```
    return a;
```

2. MergeSort method is executed. Checks if left subarray is smaller than right subarray. If it is, it will continue sorting. If it is not, it will stop since it has one element.

```
    if (indexLeft < indexRight) {
```

3. The middle index is calculated and found. This is used to split the array in two for left and right subarrays

```
        //Math for finding middle
        int rightMinusLeft = indexRight - indexLeft;
        int indexMiddle = indexLeft + rightMinusLeft / 2;
```

4. mergeSort method is used to sort left array by sending the unsorted array, left-most index position and middle index position

```
        mergeSort(a, indexLeft, indexMiddle);
```

5. mergeSort method is used to sort the right array by sending the unsorted array, middle index position + 1 and right-most index position.

```
        mergeSort(a, indexLeft: indexMiddle+1, indexRight);
```

1 is added to middle index to not include the element in middle position since the middle element is used when dividing the left part of the array. This is the same reason for all indexMiddle + 1's in the code.

6. The divided left and right array are merged together with merge method

```
        merge(a, indexLeft, indexMiddle, indexRight);
```

7. Lastly, left and right mergers are fully sorted and merged into "a" array in "mergeRun" and is returned to the menu for printing.

```
    return a;
```

Merge method:

1. Number of mergers is increased for each run

```
    Counter.numberOfMergers++;
```

2. Left index position and right index position is calculated by using variables the method is executed with

```
    int p1 = indexMiddle - indexLeft + 1;
    int p2 = indexRight - indexMiddle;
```

3. Two temporary arrays are created for left and right subarrays

```
double[] arrayLeft = new double[p1];
double[] arrayRight = new double[p2];
```

4. 1st for loop copies elements from the "a" array into left subarray index 0 to middle index position into the left temporary subarray. This runs while I is smaller than p1 (left subarray) and increases I after each run.

```
for (int i = 0; i < p1; ++i)
    arrayLeft[i] = a[indexLeft + i];
```

5. 2nd for loop copies elements from "a" middle index +1 + j (index position in right subarray). It only copies the right half of the current subarray. This runs while j is smaller than p2 (right subarray) and increases j after each run.

```
for (int j = 0; j < p2; ++j)
    arrayRight[j] = a[indexMiddle + 1 + j];
```

6. I, j, and k are used as index position trackers. I and j is set to 0 while k is set to the current position in left subarray (indexLeft). I is for index position in left subarray, j is for index position in right subarray and k is for the current index position in the "a" array.

```
int i = 0;
int j = 0;
int k = indexLeft;
```

7. 1st while loop is set to run as long as I is smaller than p1 and j is smaller than p2.

```
while (i < p1 && j < p2) {
```

8. Total number of comparisons are increased at the start of the execution of the while loop above.

```
Counter.numberOfComparisons++;
```

The if statements runs if the left subarray is smaller or equal to the right subarray element and sets k's current position in "a" array to be the current element in left subarray. I is then increased.

```
if (arrayLeft[i] <= arrayRight[j]) {
    a[k] = arrayLeft[i];
    i++;
}
```

If the above "if" fails, the else section will trigger and set k's current position in "a" array to be the current element in right subarray. J is then increased.

```
else {
    a[k] = arrayRight[j];
    j++;
}
```

At the end of the while loop, k is increased since the current index position is merged and must move to the next index position (element)

```
k++;
```

9. The 2nd while loop runs as long as value I is smaller than p1 (left subarrays index)

```
while (i < p1) {
```

This loop sets "a" array's current index position (k) to be the value in left subarray index position (i).

```
a[k] = arrayLeft[i];
```

I and k is then increased before the while runs again until I is bigger than p1 (left subarrays index)

```
i++;
k++;
```

10. The 3rd while loop runs as long as value j is smaller than p2 (right subarrays index)

```
while (j < p2) {
```

This loop sets "a" array's current index position (k) to be the value in right subarray index position (j).

```
a[k] = arrayRight[j];
```

J and k is then increased before the while runs again until J is bigger than p2 (right subarrays index)

```
j++;
k++;
```

Visualization

Array used for visualization:

Data values	1	4	3	2
Index	0	1	2	3

Divide and conquer				
Initial array	1432			
Divide section				
1st divide	1, 4			
2nd divide	1	4		
3rd divide			3, 2	
			3	2
Conquer and merge section				
Divided array	1	4	3	2
1st merge	1, 4			
2nd merge			2, 3	
3rd merge	1, 2, 3, 4			
Sorted array	1, 2, 3, 4			

The end array after sorting is 1 2 3 4. With a bigger array, the divide section and conquer and merge section will be longer but will still be sorted when the algorithm has fully executed.

Mean average time usage

When starting the program, we have created 2 random algorithms that are executed and ignored before using timing to calculate a mean average for each algorithm.

Run	Version	Time (seconds)	Swaps	Comparisons	Mergers
1	Merge sort	0.2584	0	599	110
2	Merge sort	0.2015	0	599	110
3	Merge sort	0.1179	0	599	110
4	Merge sort	0.1134	0	599	110
5	Merge sort	0.1152	0	599	110
	Mean average	0.16128	0	599	110
1	Merge sort Randomized	0.222299	0	620	110
2	Merge sort Randomized	0.1767	0	623	110
3	Merge sort Randomized	0.24	0	614	110
4	Merge sort Randomized	0.147	0	616	110
5	Merge sort Randomized	0.1692	0	617	110
	Mean average	0.191040	0	618	110

Computational complexity

Time Complexity

Merge Sort's time complexity is based on the temporary merged arrays, which are arrayLeft and arrayRight. The method to calculate this is $O(n\ log_2\ n)$ where n is the number of elements in array and $log_2\ n$ is how many times the array is divided in half until each of the subarrays consists of one element. We use log_2 since merge sort divides arrays in half (Animesh, n.d.-b).

Best, average and worst case is $O(n \log n)$.
To calculate the time complexity for merge sort it is recommended to use $O(N\ *\ log2N)$ which outputs the same as $O(n \log n)$ (Animesh, n.d.-b).
The calculation of these cases are the same with only unique values:

Best, average and worst case
$111 *\ log_2(111) = 111 * 6.79 \approx 754$

Running merge sort on a randomized array will not change the time complexity. The algorithm will still need to divide 111 elements into subarrays containing one element and merge them together again. The number of mergers it must run is always n – 1, as seen in the

"Mean average time used" above. In this algorithm, it is always 110 splits and mergers, since it must first be split up and then merged to fully rebuild the sorted array (Animesh, n.d.-b; kartik, n.d.-c).
So no, the time complexity and merge operations does not change if you randomly shuffle the list before sorting.

Space complexity

Merge sort's space complexity is $O(n)$. It uses an external array to merge the finished sorted array halves which is also used to store the merged results. N is the size of the external input array. It only uses temporary array and does not care about the extra temporary variables, it only counts arrayLeft and arrayRight (Animesh, n.d.-b).

When to use

Merge sort is recommended to be used on larger datasets since it is a stable algorithm, and its time complexity tells us it will perform well on larger datasets. It is generally easy to implement in code, which is shown in the MergeSort.java file.
It does, however, require extra memory for storage of divided and halved arrays when it runs its sorting process. This algorithm is not as quick as quick sort since quick sort is able to use cache. It is, however, better than a lot of other algorithms when used with larger datasets. For our dataset of 111 unique alcohol values, it is a good choice since it is stable, and time complexity makes it future proof for usage on the whole non-unique dataset (Animesh, n.d.-b; kartik, n.d.-c).

Problem 4 – QuickSort

Quick Sort is a sorting algorithm based on the Divide and Conquer that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array (Gupta, 2026e, p. 12; kartik, n.d.-d).
When a pivot is selected, the array will be partitioned into two sub-arrays. The values smaller than the pivot will be placed in one and the values larger will be placed in the other (Rashmi, 2026).

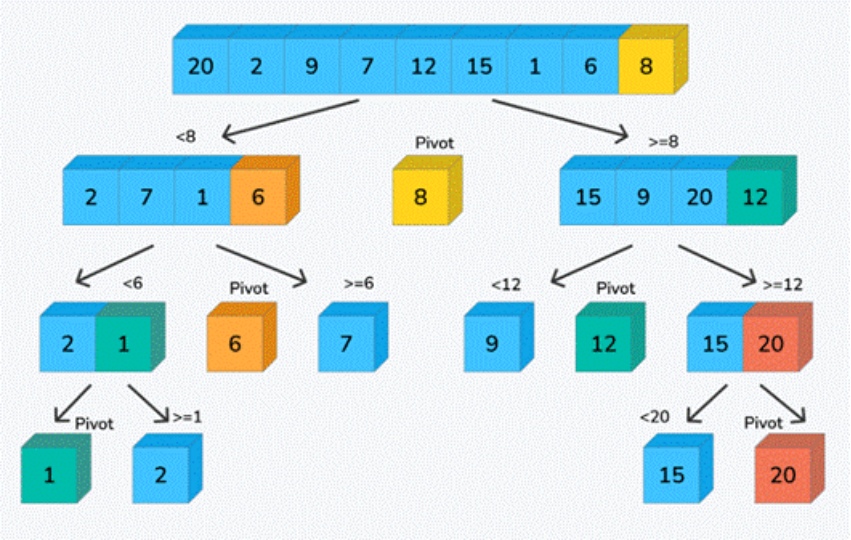


Figure 3 https://workat.tech/problem-solving/tutorial/sorting-algorithms-quick-sort-merge-sort-dsa-tutorials-6j3h98lk6j2w

In our code, Quick Sort can choose the pivot in four different ways:

1. first element as pivot
2. last element as pivot
3. random element as pivot
4. median of three as pivot

Implementation

Sorting

```
static void sorting(Double[] a, int first, int last, char sortOrder) {

    // Only sort if its more than one element
    if (first < last) {
```

This is the self-calling Quick Sort function. It may only work if the sub-array has at least 2 elements.

```
// picks up the pivot so it can move the elements around it
int pi = partition(a, first, last, sortOrder);

// Recursively sort left side
sorting(a, first, last: pi - 1, sortOrder);

// Recursively sort right side
sorting(a, first: pi + 1, last, sortOrder);
```

It will then locate the pivot and move the elements around it. It then starts to move all the values that are lower than the pivot to the left of the array. Then it moves all the values that are higher than the pivot to right of the array.

Partition

```
static int partition(Double[] a, int first, int last, char sortOrder)

    // Get pivot index based on chosen strategy
    int indexPivot = pivot(a, first, last, sortOrder);
```

Here the pivot index is selected depending on your chosen strategy.

```
// Move pivot to the end of the array (if not already there)
if (indexPivot != last) {
    swap(a, indexPivot, last);
    Counter.numberOfSwaps++;
}

// Pivot value (now at last position)
double pivot = a[last];

// n marks the boundary for elements smaller than pivot
int n = first - 1;

// Loop through elements
for (int i = first; i < last; i++) {
```

Swaps the pivot at the end of the array. It makes a variable which is the boundary for the array that separates the right side with bigger values and the left side with lower values.

```
// Loop through elements
for (int i = first; i < last; i++) {

    Counter.numberOfComparisons++;

    // If element is smaller than pivot
    if (a[i] < pivot) {
        n++;

        // Swap to move smaller element to the left side
        if (i != n) {
            swap(a, i, n);
            Counter.numberOfSwaps++;
        }
    }
}
```

The for loop, loops true all the elements. The first if loop checks if the value is lower than pivot. If yes, it then increases n to make sure the new element on the left has space and incise the left arrays length. The second if loop makes sure element is moved. i is the current location of the element and n is the new location of the element. The swap command will swap the location of the variable to the left of the array.

```
// Place pivot in correct sorted position
int k = n + 1;
if (k != last) {
    swap(a, k, last);
    Counter.numberOfSwaps++;
}

// Return pivot index
return n + 1;
```

Now all elements that are lower than pivot are moved to the right of the array, but the index is still at the end of the array. The code simply places the pivot on n+1. Then the code swaps the pivot in place, and the pivot separates the two halves of the array.

Swap

```
// Swap two elements in the array
private static void swap(Double[] a, int i, int j) {  3 usages
    double temp = a[i];
    a[i] = a[j];
    a[j] = temp;
}
```

The swap function makes it easy to swap elements around the array. It starts with making a temporary variable that saves the value at position i. then move value from j to i. but the original value inside temp to j. This makes the values are swapped, now I has the value of j and vice versa.

Pivot

```
// Method to choose which element to use as pivot
static int pivot(Double[] a, int first, int last, char sortOrder) {  1 usage
    switch (sortOrder) {
```

This function is where the pivot is chosen, and it returns the index of the element used as pivot. The sort Order variable tells which pivot strategy to use. Switch (sortOrder) decides with order will be used.

```
// Use first element as pivot
case 'a' -> {
    return first;
}

// Use last element as pivot
case 'b' -> {
    return last;
}

// Use a random element as pivot
case 'c' -> {
    int random = new Random().nextInt( bound: last - first + 1) + first;
    return random;
}

// Use median of three (first, middle, last) as pivot
case 'd' -> {
    int middle = ((first + last) / 2);
    int empty = 0;
```

In case a, you use the first element as pivot.
In case b, you use the last element as pivot.
In case c, you use a random element as pivot.

In case d, you use the median of the three.

```
// Check which value is in the middle (median)
if ((a[middle] <= a[last] && a[middle] >= a[first])) {
    return middle;
}
if (a[middle] >= a[last] && a[middle] <= a[first]) {
    return middle;
}
```

Case d now has three values: first, last and middle. We want the median value. We need to make sure that the value is between first and last.
In the first if we see, a[first] ≤ a[middle] ≤ a[last], then a[middle] is the median. It returns the middle.
In the second if we can se that the last now has less value than the middle and first, a[last] ≤ a[middle] ≤ a[first], here a[middle] is still the median. It returns the middle.

```
if (a[first] <= a[last] && a[first] >= a[middle]) {
    return first;
}
if (a[first] >= a[last] && a[first] <= a[middle]) {
    return first;
}
```

In the third if we can see that middle is lower value than bout first and last. This means it look like this: a[middle] ≤ a[first] ≤ a[last] and this mean the median is now first so it returns first. We can see the same in the Fourt if that the first is in-between last and middle like this: a[last] ≤ a[first] ≤ a[middle]. This means the median will return first.

```
if (a[last] <= a[first] && a[last] >= a[middle]){
    return last;
}
if (a[last] >= a[first] && a[last] <= a[middle]) {
    return last;
}
```

In the fifth and sixth if loops we can see that the last is in-between the others like this: a[first] ≤ a[last] ≤ a[middle] and like this: a[middle] ≤ a[last] ≤ a[first].

Order	Median	Code returns

first ≤ middle ≤ last	middle	return middle
last ≤ middle ≤ first	middle	return middle
middle ≤ first ≤ last	first	return first
last ≤ first ≤ middle	first	return first
first ≤ last ≤ middle	last	return last
middle ≤ last ≤ first	last	return last

Visualization

Array used for visualization:

Data values	1	4	3	2
Index	0	1	2	3

We will use the last element as pivot so the value will be 2

Pass	Compared element	Array before	Array after	Swap
1	1	1, 4, 3, 2	1, 4, 3, 2	1 is smaller than pivot, stays left
2	4	1, 4, 3, 2	1,3,2,4	4 is greater than pivot and moves to the right
3	3	1,3,2,4	1,2,4,3	3 is greater than pivot and moves to the right
4	3	1,2,4,3	1,2,3,4	Both sides of the pivot get sorted, 1 is unchained it is alone, but 3 and 4 gets sorted
end	Pivot = 2	1,2,3,4	1,2,3,4	Pivot in the correct position

After the pivot splits the array into a left part which is smaller and a right part which is bigger, but sides will be sorted 1 is alone so already sorted, on the right 3 and 4 will sort.

Mean average time usage, Swaps, and comparisons

When starting the program we have created, 2 random algorithms are run and ignored before using timing to calculate a mean average for each algorithm.

Run	Version	Time (seconds)	Swaps	Comparisons	Mergers
1	Quick Sort first pivot	0.5251	323	740	0
2	Quick Sort first pivot	0.0838	323	740	0
3	Quick Sort first pivot	0.5063	323	740	0
4	Quick Sort first pivot	0.0823	323	740	0
5	Quick Sort first pivot	0.5214	323	740	0
	Mean average	0.34378	323	740	0

Run	Version	Time (seconds)	Swaps	Comparisons	Mergers
1	Quick Sort last pivot	0.2759	307	685	0
2	Quick Sort last pivot	1.8887	307	685	0
3	Quick Sort last pivot	0.7893	307	685	0
4	Quick Sort last pivot	0.4282	307	685	0
5	Quick Sort last pivot	0.3088	307	685	0
	Mean average	0.73818	307	685	0

Run	Version	Time (seconds)	Swaps	Comparisons	Mergers
1	Quick Sort random pivot	0.7592	327	659	0
2	Quick Sort random pivot	0.8083	327	859	0
3	Quick Sort random pivot	0.2643	327	676	0

4	Quick Sort random pivot	0.4793	327	759	0
5	Quick Sort random pivot	0.4486	327	776	0
	Mean average	0.55194	327	745.8	0

Run	Version	Time (seconds)	Swaps	Comparisons	Mergers
1	Quick Sort median pivot	0.3038	373	665	0
2	Quick Sort median pivot	0.5222	373	665	0
3	Quick Sort median pivot	0.5343	373	665	0
4	Quick Sort median pivot	0.5815	373	665	0
5	Quick Sort median pivot	0.3261	373	665	0
	Mean average	0.45358	373	665	0

Results:

Pivot Version	Time	Swaps	Comparisons	Mergers
Quick Sort first pivot	0.34378	323	740	0
Quick Sort last pivot	0.73818	307	685	0
Quick Sort random pivot	0.55194	327	745.8	0
Quick Sort median pivot	0.45358	373	665	0

Here we can see that the number of comparisons changes depending on the pivot strategy. The median pivot has the least amount while random has the most. Less comparisons, more balanced split, uneven split more comparisons.

The data shows that the median pivot clearly is the best pivot selection strategy for our dataset. With the least amount of comparisons, it clearly is the one with the most balanced split and is closest to best case time complexity.

Computational complexity

Time Complexity

Best case: O(n log n)
The best case happens when the pivot divides the array into two nearly equal parts every time. This gives a balanced recursion tree and makes the algorithm efficient.

Average case: O(n log n)
In the average case, Quick Sort also performs very efficiently. Even if the partitions are not perfectly equal, they are often balanced enough that the algorithm still runs in O(n log n) time.

Worst case: ($O(n^2)$)
The worst case happens when the pivot always ends up being the smallest or largest element. Then one partition becomes very small and the other becomes very large. This creates unbalanced recursion and much more work (Geeks for Geeks, 2025).
For example, if the array is already sorted and the first or last element is always chosen as pivot, Quick Sort can perform poorly: W(n)<=n(n-1)/2 = O(n2) (When pivot is always the smallest or largest element) (Gupta, 2026e, p. 13).

Space Complexity

The Quick Sort algorithm itself is usually considered an in-place sorting algorithm because partitioning is done inside the same array.
Best case scenario: as a result of balanced partitioning leading to a balanced recursion tree with a call stack of size O(log n).
Worst case scenario: due to unbalanced partitioning leading to a skewed recursion tree requiring a call stack of size O(n) (kartik, n.d.-d).

When to use

Benefits of Using Quicksort:
Quicksort is widely used because of its efficient average-case performance, running in O(n log n) time, which makes it fast for most practical situations. It is an in-place sorting algorithm, meaning it requires only a small amount of additional memory, typically O(log n) due to recursion. This makes it space-efficient compared to some other sorting methods. Another advantage is its high performance on large datasets, where it often outperforms many other algorithms in practice. Quicksort is also cache-friendly, as its memory access patterns tend to make good use of the CPU cache, improving overall speed. It is a versatile algorithm that can handle a wide range of input data and, with proper pivot selection, performs well across different input distributions.
Additionally, Quicksort can be parallelized because of its divide-and-conquer approach, allowing it to take advantage of multi-core processors. It is especially well-suited for sorting arrays, although it is generally less effective for linked lists (TheKnowledgeAcademy, 2025).

Drawbacks of Quicksort:
Quicksort has several disadvantages despite its strong performance in many cases. One major issue is its worst-case time complexity of $O(n^2)$, which can occur when the pivot is poorly chosen, such as when the input data is already sorted or highly unbalanced. It is also not a stable sorting algorithm, meaning it does not preserve the relative order of equal elements. Another concern is its reliance on recursion, which can lead to deep recursive calls and potentially cause a stack overflow for very large datasets. The efficiency of Quicksort is highly dependent on good pivot selection, making it sensitive to how the pivot is chosen. For small datasets, Quicksort can be less efficient due to its overhead, and simpler algorithms like insertion sort may perform better.
Additionally, frequent swapping of elements can add overhead and reduce performance in some cases. Quicksort may also perform poorly on skewed or already sorted data unless optimizations, such as using random pivots, are applied (TheKnowledgeAcademy, 2025).

Overall, Quicksort is a powerful and efficient sorting algorithm that performs very well on large datasets and in average-case scenarios, especially when memory efficiency and speed are important. However, its performance can degrade significantly in the worst case, and it requires careful implementation to avoid issues like poor pivot selection and deep recursion. While it is one of the fastest sorting algorithms in practice, its drawbacks mean that it is not always the best choice for every situation.

References

Animesh. (n.d.-a). *Time and Space Complexity Analysis of Bubble Sort*. GeeksforGeeks. Retrieved April 17, 2026, from https://www.geeksforgeeks.org/dsa/time-and-space-complexity-analysis-of-bubble-sort/

Animesh. (n.d.-b). *Time and Space Complexity Analysis of Merge Sort*. GeeksforGeeks. Retrieved April 17, 2026, from https://www.geeksforgeeks.org/dsa/time-and-space-complexity-analysis-of-merge-sort/

Brilliant. (n.d.). *Insertion Sort | Brilliant Math & Science Wiki*. Brilliant. Retrieved April 17, 2026, from https://brilliant.org/wiki/insertion/

Exponent. (n.d.). *Insertion Sort Algorithm | Interview Cake*. Programming Interview Questions and Tips. Retrieved April 17, 2026, from https://www.interviewcake.com/concept/java/insertion-sort

Gupta, R. (2026a, January 8). *Learning Outcome 1 Basics of Data Structures and Algorithms*.

Gupta, R. (2026b, January 8). *Learning Outcome 1 Basics of Data Structures and Algorithms* [Lecture presentation].

Gupta, R. (2026c, January 15). *Learning Outcome 1 Non-Primitive Linear Data Structure*.

Gupta, R. (2026d, January 29). *Learning Outcome 5 Understanding Algorithm Efficiency*.

Gupta, R. (2026e, March 6). *Learning Outcome 3 Sorting Algorithms*.

Gupta, R. (2026f, March 24). *PG4200-H: Data Structures and Algorithms*. Kristiania University College.

kartik. (n.d.-a). *Bubble Sort*. GeeksforGeeks. Retrieved April 17, 2026, from https://www.geeksforgeeks.org/dsa/bubble-sort-algorithm/

kartik. (n.d.-b). *Insertion Sort Algorithm*. GeeksforGeeks. Retrieved April 17, 2026, from https://www.geeksforgeeks.org/dsa/insertion-sort-algorithm/

kartik. (n.d.-c). *Merge Sort*. GeeksforGeeks. Retrieved April 17, 2026, from https://www.geeksforgeeks.org/dsa/merge-sort/

kartik. (n.d.-d). *Quick Sort*. GeeksforGeeks. Retrieved April 17, 2026, from https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/

Marculescu, B. (n.d.). *Lesson 03: Runtime Analysis and Sorting*. Kristiania University of Applied Science.

Neapolitan, R. E. (2025). *Foundations oF Algorithms* (5th ed.). Jones & Bartlett Learning.

Noroff. (n.d.). *Insertion Sort*. Noroff. Retrieved April 17, 2026, from https://mollify.noroff.dev/content/feu2/javascript-2/module-5/algorithms/sorting-and-searching/insertion-sort?nav=undefined&utm_source=chatgpt.com

priyavermaa1198. (n.d.). *Difference between JIT and JVM in Java*. GeeksforGeeks. Retrieved April 17, 2026, from https://www.geeksforgeeks.org/java/difference-between-jit-and-jvm-in-java/

rajput-ji. (n.d.). *Difference and similarities between HashSet, LinkedHashSet and TreeSet in Java*. GeeksforGeeks. Retrieved April 17, 2026, from https://www.geeksforgeeks.org/java/difference-and-similarities-between-hashset-linkedhashset-and-treeset-in-java/

Subject description PG4200 Algorithms and Data Structures. (n.d.). Retrieved April 17, 2026, from https://www.kristiania.no/en/syllabus/school-of-economics-innovation-and-technology/first-cycle-degree/pg4200/algorithms-and-data-structures/

TheKnowledgeAcademy. (2025, December 12). *Quick Sort Algorithm: Everything You Should Know*. TheKnowledgeAcademy. https://www.theknowledgeacademy.com/blog/quick-sort-algorithm/